BeyondPPA: Human-Inspired Reinforcement Learning for Post-Route Reliability-Aware Macro Placement

Ishraq Tashdid[1], Valentina Terry[1], Jordan Merkel[1],
Tasnuva Farheen[2], Sazadur Rahman[1]
[1]Department of Electrical and Computer Engineering, University of Central Florida, Orlando, FL, USA
[2]Department of Electrical and Computer Engineering, Louisiana State University, Baton Rouge, LA, USA
{ishraq.tashdid, valentina.terry, jordan.merkel, sazadur.rahman}@ucf.edu[1], tfarheen@lsu.edu[2]

***Abstract*—Macros are pre-designed IP blocks—including memory arrays, analog circuits, and custom accelerators—that occupy large, fixed-shaped regions within a chip layout. Their placement critically impacts system-on-chip (SoC) reliability by affecting power integrity, clock distribution, and signal robustness. Despite advances in commercial EDA tools, macro placement remains a challenging and manual task, often leading to routing congestion, IR drop, and clock skew. While reinforcement learning (RL) techniques show promise, they frequently lack the structural intuition and constraint-awareness seen in expert designs. To bridge this gap, we propose *BeyondPPA*, a human-inspired RL placement framework that integrates practical layout heuristics into the agent's reward function—including macro-grid alignment, IO keepout enforcement, corner biasing, and density balancing. These spatial priors guide the agent toward producing more routable and reliable layouts. On the ISPD benchmark suite, *BeyondPPA* reduces clock latency by up to 21.3%, clock uncertainty by 23.2%, average IR drop by 57.2%, EM violations by 30.4%, and glitch voltage by $9.2\times$ compared to state-of-the-art baselines. These gains highlight that embedding designer-inspired rules into RL not only sustains competitive PPA metrics—achieving 1.43% wirelength and 3.93% congestion reduction—but also significantly improves post-routing reliability, essential for manufacturable SoCs.**

***Index Terms*—Chip placement, Reinforcement learning.**

I. Introduction

Modern integrated-circuit (IC) design now juggles billions-node netlists and heterogeneous blocks [1], from standard-cell logic to fixed-shape 'macros' such as SRAMs, analog blocks, and domain-specific accelerators. Electronic-design-automation (EDA) flows tackle this scale with staged automation [2, 3] - logic synthesis, floorplanning, placement, clock-tree synthesis, and routing, but macro placement sets the tone for everything that follows [4]. Because macros occupy large die regions, prohibit overlap, and constrain routing channels, their locations dominate a chip's eventual power, performance, and area (PPA). Unlike standard cells they are essentially immovable once placed, so early choices are decisive. The search space, however, is vast, placing thousands of non-uniform rectangles on a fine grid is NP-hard even after aggressive abstraction [5], and a single industrial sign-off run can consume hours to days [4]. Sub-optimal decisions ripple forward by creating blockages, stretching interconnects, inducing congestion, and triggering timing violations and manufacturability [6].

A broad spectrum of macro-placement engines exists today (Table I), ranging from analytical optimizers [7, 8] and clustering heuristics [9, 10] to recent learning-based schemes that treat the task as search or decision-making [4, 11–14]. While these tools efficiently minimize wirelength and congestion, they often fall short of the disciplined, "clean-grid" layouts produced by expert floorplanners [15, 16]. The gap is apparent on real designs. On the widely studied Ariane-133 benchmark, for instance, DREAMPlace variants [7, 17] frequently leave skewed macros, empty corners, and blocked I/O channels [15, 16, 18, 19]. These artifacts trigger a cascade of penalties: ❶ 3% longer wires and localized congestion that worsen post-routing [15]; ❷ misaligned macro grids that hinder legalization, clock synthesis, and routing, prompting manual rework; ❸ wasted silicon in unused corners, inflating die area and reducing substrate reliability; and ❹ blocked I/O rings that extend pad routes and hurt signal integrity (SI) [6]. These limitations highlight the persistent gap between automated macro placers and the nuanced reasoning of expert designers. While existing tools optimize abstract objectives, they often ignore structural layout features critical to regularity, reliability, and testability [6]—motivating a shift toward placement methods guided by human-like heuristics.

TABLE I: Comparison of this work with existing macro placement methods.

Related Works	Agent	Target Features	
		Performance	Structural
GraphPlace [4]	RL	HPWL	×
DP [7]	DL	HPWL	×
ChipFormer [13]	RL	HPWL, Cong.	×
EfficientPlace [14]	MCTS+RL	HPWL	×
MaskPlace [11]	RL	HPWL, Cong.	×
WireMask [12]	RL	HPWL, Cong.	×
BeyondPPA	**DP+RL**	**HPWL, density**	**IO keepout, notch, align**

DP: DREAMPlace, RL: Reinforcement learning, DL: Deep learning.
HPWL: Half perimeter wirelength, MCTS: Monte Carlo Tree Search.

More specifically, we explore the following research questions in this work to further investigate the gap and potential resolution.

RQ1. How to formalize human-like macro placement aspects, such as precise macro grid alignment, IO keepout zones, standard cell continuity, and uniform density into structural learning features?

RQ2. How can human-inspired structural features be systematically integrated into a reinforcement learning framework to dynamically balance conflicting requirements, such as minimizing wirelength versus notch removal, to achieve optimal placement outcomes?

RQ3. Can RL agents learn these features performing human-like macro placement to improve routing reliability while preserving PPA?

By researching the above questions, in this paper, we propose *BeyondPPA*, a human-inspired structural feature for RL-based macro placement focusing reliability. Our contributions are as follows.

- We review existing macro placement methods, highlight key gaps relative to industry needs, and define novel structural features - IO keepout, notch removal, macro alignment, density, and wirelength to better align with practical and reliable IC design. We formalize these features into novel constraints and cost functions to replicate human-like macro placement aspects (Sec. III-A).
- We propose *BeyondPPA*, a bi-step macro placement method - a global macro placement by DP [7] and detail placer by RL-agent that mimics a human engineer optimizing a novel reward function based on the defined constraints (Sec. III-B). *BeyondPPA* can automatically reprogram the features by dynamically choosing the appropriate weights and biases to balance conflicting requirements.

979-8-3315-3762-3/25/$31.00 © 2025 IEEE

TABLE II: Impact of Macro Placement and Routing Shapes on Different Chip Reliability Metrics

Reliability Metrics	Impacting Placement and Routing Factors	Typical Failure Symptoms
RC delay and slew	• Macro spacing & channel width • Extra wire-length from routing detours • Macros that force long pin escapes across rows	Timing violations, clock jitters
Crosstalk Noise	• Long, parallel detours created by meandering routing • Tight macro spacing packs victim-aggressor nets together • Deep notches that trap adjacent nets in narrow corridors	Glitches, timing delay
PDN IR-drop (static & dynamic)	• High-current macros placed far from top-metal grid • Narrow channels between macros limiting strap width	Timing violations, momentary logic resets
Electromigration (EM) & via-EM	• Routing detours → longer lines & tall via stair-cases • Notches that concentrate current in few wires	Gradual opens/shorts, sudden field failures
Thermal Gradient	• Clustering high-power macros without spacing • Dense meanders that act as resistive heaters	Local delays, accelerated EM/BTI aging, skew drift
Antenna Violations	• Long dangling routes caused by macros spacing • Extra jogs from notches	Gate-oxide punch-through, random opens
Aging (OCV / AOCV / POCV)	• Longer wire-length and deeper logic caused by detours • Temperature peaks from poor macro spacing	Timing failures overtime, SRAM access errors
Design for manufacturability	• Tight macro spacing shrinks spacing margin • Detours due to misalignment and I/O blockage	Lower fab yield; random open/bridge defects

- ***To the best of our knowledge, this is the first work to report detailed post-routing reliability metrics, providing a more realistic evaluation of physical design quality.*** On the ISPD benchmark suite [15, 20], *BeyondPPA* achieves significant reliability improvements, including an average reduction of 21.3% in clock latency, 23.2% in clock uncertainty, 57.2% in average IR drop, 30.4% fewer EM violations, and a 9.2× decrease in glitch voltage compared to state-of-the-art baselines.

The paper is organized as follows: Sec. II outlines macro placement goals and the motivation for human-inspired reliability. Sec. III details the proposed *BeyondPPA* framework and RL formulation. Sec. IV presents comparative results against prior works, followed by Sec. V which analyzes key findings and limitations. Finally, Sec. VI concludes the paper with a summary and future directions.

II. Background and Motivation

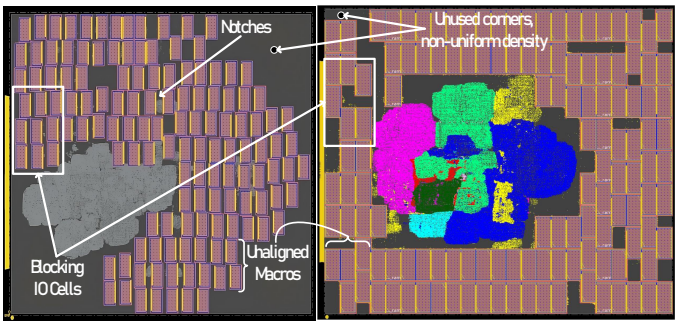
In this section we discuss the fundamentals of macro placement objectives, existing macro placement methods and their limitations, human aspects in macro placement, and our motivation.

A. Fundamentals of Macro Placement

As discussed in Sec. I, macro placement is the linchpin between floorplanning and detailed physical synthesis in an ASIC flow. A careless arrangement inflates half-perimeter wire-length (HPWL), breeds routing-congestion hot spots, and skews density, complicating timing closure, routing quality, such as, signal integrity, IR-drop, crosstalk noise, and clock-tree reliability. A disciplined, grid-aligned arrangement does the opposite - it shortens nets, eases global routing, and simplifies clock-tree synthesis. Accordingly, placement quality is gauged by HPWL, congestion, and density - ideally the first two are minimized while utilization remains uniform across the core. Shrinking nodes and heterogeneous IP only sharpen these demands, tying good macro placement not just to PPA but also to manufacturability and long-term reliability and calling for algorithms that can traverse vast search spaces under tight, foundry-driven constraints.

B. Existing Macro Placement Techniques and Their Limitations

Recent advancements in IC design have modeled macro placement as a learning problem [4, 7] and incorporated DL/RL methods to improve placement optimization. Traditional approaches, e.g., DREAMPlace (DP) [7] and Hier-RTLMP [9], employ analytical and clustering methods to place macros. AutoDMP [8] and its spin-off IncreMacro [10] utilizes multiobjective tree-structured parzen estimator-based parameter tuning. On the other hand, Google's GraphPlace (*CT*) [4] introduces RL-based macro placement using legacy design data and iteratively optimizing a cost function derived



(a) Analyhtical Methods (DREAMPlace) (b) RL-based Methods (GraphPlace)

Fig. 1: Limitations of existing macro placers in Ariane-133 benchmark - (a) DREAMPlace [7], (b) GraphPlace [4].

from wirelength, congestion, and density as proxy metrics. Since then, several RL-based macro placement methods have been proposed by leveraging offline decision transformer [13], visual representation learning of circuit modules [11], monte carlo tree search [14], and wire-mask guided black box optimization [12] which often do not align with industry practices [15, 16] (See Fig. 5). Researchers have highlighted the deficiency of existing learning-based macro placers compared to human experts for large macro-heavy designs [15]. On the contrary, analytical placers, like DREAMPlace [7], generate better-routed wirelength than *CT* [4]; however, they risk creating macro blockage due to mixed-size placement [15, 16]. Fig. 1(a) and (b) shows the macro placement layout of *Ariane* [18] benchmark using existing analytical methods (DREAMPlace [7]) and RL-based methods (GraphPlace [4]). Both approaches suffer from - *(i) unused corners, (ii) non-uniform density, (iii) notches (standard cells trapped in narrow channels between macros), (iv) unaligned macros, and (v) I/O port blocking which creates numerous reliability and PPA issues*. Table II presents several reliability metrics, corresponding placement and routing factors, and their impact on design failures [6]. ❶ Unaligned macros force routing detours impacting SI (crosstalk noise, PDN IR-drop), and EM violations [6]. ❷ Most of the existing approaches do not honor a keepout region for I/O ports as highlighted in Fig. 1(a), causing degraded signal quality [15]. ❸ Mixed-size placement by analytical placers [7, 21] leave unused corners at the layout leading to non-uniform density, causing congestion. ❹ Existing methods leave notches between macros, causing local congestion, and detours impacting clock tree, SI, EM, and antenna violations [6]. ❺ Finally, macro channel spacing impacts several reliability metrics in Table II causing numerous design failures. The following subsection discusses how these limitations and the approaches followed by human engineers motivate this work.

C. Motivation: Human-like Macro Placement for Chip Reliability

In IC design, human experts often follow a distinct set of macro placement strategies grounded in principles of spatial regularity, and uniformity that are mostly underrepresented in automated algorithms. ❶ Human designers tend to align macros along imaginary grid lines reserving buffer spaces around critical regions such as I/O ports [6]. This practice ensures routing uniformity, simplifies clock tree synthesis, enables predictable signal propagation [22], facilitates the clean insertion of physical buffers and protects signal integrity. ❷ Moreover, designers proactively avoid creating notches that are known to cause routing congestion, timing, and EM violations [6]. Previous works used macro halos or blockages to limit standard cell placement in macro channels to avoid creating notches [8]. ❸ Another consideration is maintaining standard cell continuity, and uniform density to limit congestion and reliability concerns [6].

Recent reinforcement learning (RL)-based placement methods have demonstrated promising gains in PPA metrics such as wirelength, density, and congestion. However, they typically optimize generic cost functions that lack structural awareness, leading to placements that diverge from expert intuition and causing routing reliability concerns. As shown in prior works [15, 16] and demonstrated in Fig. 1 and Table II, these methods frequently produce designs that degrade layout quality and reliability. This gap motivates the need for a placement method that captures human-inspired design practices while leveraging the adaptability of learning-based algorithms discussed in the next Sec. III.

III. Methodology

This section outlines the macro placement framework used in this study. We describe the structural feature extraction techniques—including alignment, IO keepout, notch elimination, congestion mitigation, and density balancing—followed by an overview of the RL architecture and implementation flow.

A. Reliability Problem Definition and Human Feature Formulation

Building on the motivation presented in Sec II-C, this work will now systematically bridge the gap between high-level placement intent and downstream reliability outcomes. While traditional macro placement has primarily focused on optimizing power, performance, and area (PPA), growing design complexity and tighter reliability budgets in modern ICs demand a more holistic view. Specifically, macro-level decisions made early in the floorplanning process can have far-reaching consequences on post-route reliability metrics such as EM, IR drop, and clock distribution robustness.

Suboptimal placements, such as poorly aligned macros, high-density clusters, or macros encroaching on critical IO regions can amplify routing congestion, disrupt power grid uniformity, and degrade timing paths due to skew or excessive insertion delays. These reliability risks, though often latent in early design stages, manifest prominently in the post-routing phase, making it crucial to embed reliability-awareness directly into the optimization process.

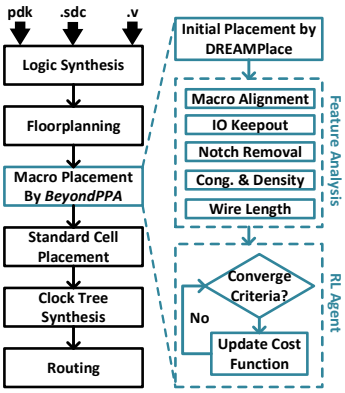


Fig. 2: *BeyondPPA* framework with structural features that are automatically configured by the RL agent for reliability-aware macro placement.

To this end, we define the macro placement problem as a constrained structural optimization task. Let $\mathcal{M} = \{M_1, M_2, \ldots, M_N\}$ represent a set of N pre-placed macros, where each macro M_i is assigned a legal coordinate (x_i, y_i) on the 2D layout canvas. The objective is to determine a placement configuration $\{(x_i, y_i)\}_{i=1}^{N}$ that minimizes a total cost function J—a weighted composition of spatial features empirically tied to physical design quality and reliability.

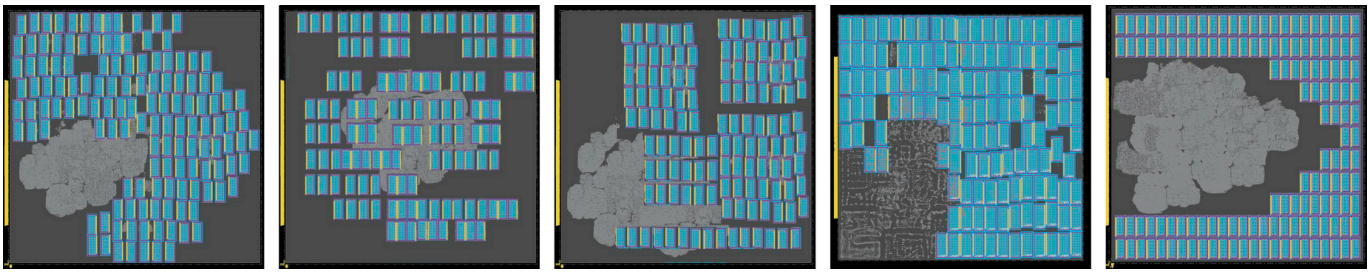
To operationalize this framework, we define a set of human-inspired cost components that guide the agent toward structurally clean and reliability-aware placements. Each component corresponds to a physical or topological feature known to impact post-route phenomena such as signal degradation, current density, and voltage stability. The core ideology of *BeyondPPA* lies in bridging traditional physical design goals with post-routing reliability concerns. These cost terms, when minimized jointly, promote placements that are not only PPA-efficient but also resilient to EM-induced failures, IR drop, and clock skew. We begin by detailing each objective term and its reliability implications below.

1) Wirelength Minimization and Signal Reliability: Regardless of the design objective—be it optimizing PPA or ensuring long-term reliability—wirelength remains the most influential and widely adopted placement metric. In *BeyondPPA*, minimizing wirelength continues to serve as a foundational priority due to its direct impact on both physical quality and electrical robustness. ❶ Shorter interconnects reduce parasitic resistance and capacitance, which in turn lowers RC delays and enhances timing margins [23]. ❷ Compact routing paths help isolate nets from neighboring aggressors, reducing the likelihood of crosstalk and noise violations [24]. ❸ Shorter high-current routes also experience reduced voltage drop, mitigating IR-related failures during switching transients [25].

2) Density Uniformity and IR Drop Mitigation: Uniform macro distribution across the layout is critical not only for routability but also for maintaining electrical and thermal reliability. In *BeyondPPA*, macro clustering is discouraged as it often leads to localized current surges and voltage droop. ❶ By encouraging even spatial distribution, the design avoids concentrated power demand regions, thereby reducing IR drop and maintaining stable supply. ❷ A balanced density also promotes uniform utilization of metal layers within the power grid, mitigating current crowding and thermal hotspots [25]. ❸ From a manufacturing perspective, uniformity also enhances yield by reducing stress concentration under process variations [26].

3) IO Keepout Zones and Noise Coupling Reduction: Proper spacing around IO pins is essential to maintain clean signal boundaries and reduce reliability risks associated with switching noise and signal interference. Macros that violate IO keepout zones may obstruct buffer insertion, limit decoupling capacitor placement, or introduce capacitive coupling to sensitive nets. ❶ Enforcing IO exclusion zones isolates noisy IO transitions from critical logic and analog regions, thereby preventing noise injection. ❷ Reserved spacing enables seamless insertion of buffers, level shifters, or electrostatic discharge clamps, enhancing signal protection without layout conflicts. ❸ Ultimately, this improves signal integrity, especially near chip peripheries where fanout trees often originate and risk is elevated [24].

4) Macro Grid Alignment and Clock Tree Quality: Grid-based macro alignment plays a vital role in improving clock tree synthesis (CTS) by promoting structural regularity and reducing variability in clock distribution paths. ❶ Well-aligned macros lead to evenly distributed clock sinks, facilitating symmetric tree construction and minimizing clock skew. ❷ Aligned placements often require fewer buffer insertions, thereby reducing cumulative insertion delay and improving overall clock latency. ❸ Furthermore, macro alignment



(a) (b) (c) (d) (e)

Fig. 3: Progressive impact of individual structural features in the *BeyondPPA* framework. (a) The initial DREAMPlace [7, 17] layout shows reliability risks, including macro clustering and IO congestion. (b) Grid alignment reduces skew by enforcing spatial regularity. (c) IO keepout zones clear periphery regions, lowering coupling noise. (d) Density balancing mitigates IR drop by avoiding localized power demand. (e) Full integration of all features yields a robust layout that enhances EM resilience, IR uniformity, and clock quality.

supports clear partitioning of clock domains, enabling robust hierarchical CTS and simplifying timing closure [27, 28].

5) Notch Avoidance and Routing Resilience: Notches—narrow concave gaps formed between adjacent macros—are widely recognized as undesirable artifacts in IC layouts due to their adverse effects on signal and power routing. ❶ These tight spaces lead to routing hotspots and current crowding, thereby increasing the risk of electromigration (EM) and local thermal stress. ❷ Wires routed through notches often require sharp bends or detours, which elevate resistive losses and degrade signal timing. ❸ Additionally, notches reduce the continuity of power grid metal, weakening IR resilience and complicating voltage delivery across the chip.

For a complete breakdown of the structural cost terms—covering wirelength, density balance, IO spacing, macro alignment, and notch penalties—please refer to Appendix A, where we present the mathematical definitions supporting our reliability-driven placement.

B. RL Framework for Reliability-Aware Placement

The *BeyondPPA* methodology integrates human-inspired structural heuristics directly into a reinforcement learning (RL) framework to automate reliability-centric macro placement. Instead of optimizing generic cost functions as seen in traditional methods, this approach trains an RL agent to learn spatial patterns that improve post-route reliability outcomes such as EM robustness, IR resilience, and clock tree quality. The agent interacts with a physical layout environment, iteratively refining placements based on feature-guided reward signals derived from the structural cost function J introduced in Eq. 11.

1) State Representation: At each decision step s_t, the reinforcement learning agent observes the current macro placement configuration as a multi-channel spatial tensor. This state representation includes a binary occupancy map that encodes the locations of all placed macros on the canvas, feature maps that reflect wirelength and local density distributions, and reliability-aware masks highlighting regions such as IO keepout zones, notch-sensitive areas, and grid anchor points. Additionally, deviation fields are computed to indicate how far each macro deviates from the nearest grid location, capturing alignment irregularities. Together, these channels form a structured visual input that enables convolutional architectures to identify spatial patterns associated with reliability hazards such as electromigration, IR drop, and clock skew.

2) Action Space: At each time step, the agent executes a structured action $a_t = (M_i, x', y', \theta)$, where a macro M_i is selected and repositioned to a new legal coordinate (x', y') on the canvas with a potential orientation θ chosen from $\{0°, 90°, 180°, 270°\}$. The macro selection process is informed by placement heuristics or gradients derived from the reliability-aware cost function, prioritizing moves that mitigate violations such as IR drop clustering or notch formation.

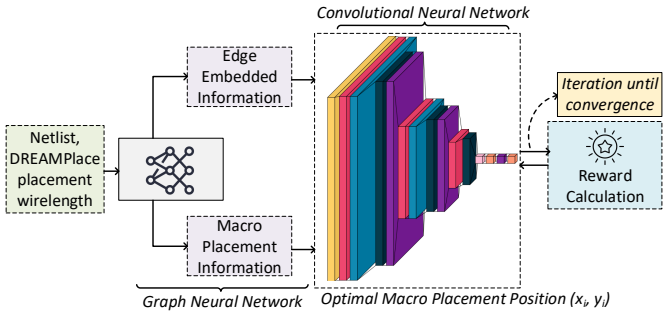


Fig. 4: Network architecture of the *BeyondPPA* reinforcement learning agent.

The updated position is chosen to target specific improvements, including reduced current density concentration or enhanced power delivery balance. The rotation angle is used to align macros with routing symmetry or facilitate signal escape paths. Through this composite action space, the agent iteratively adjusts the macro layout, progressively enhancing both structure and post-route reliability.

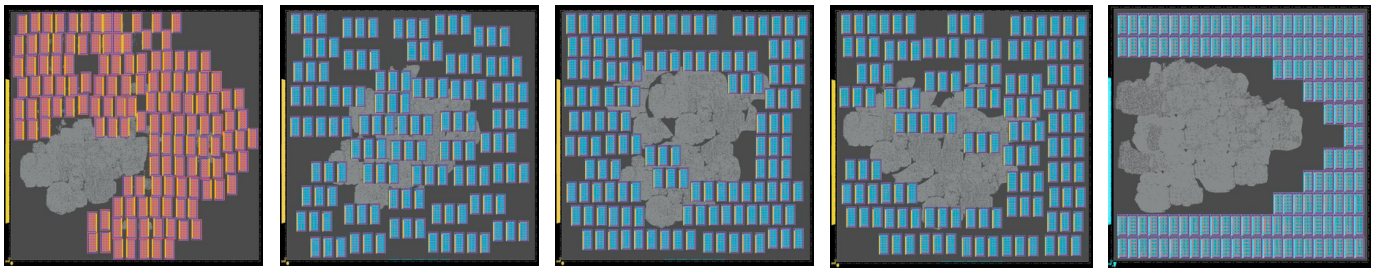
3) Reward Design: The reward function is derived directly from the structural cost function, providing real-time feedback on how each action impacts layout quality. At time step t, the reward is defined:

$$R_t = \frac{1}{1 + J_t} \tag{1}$$

where J_t (defined in Eq. 11) is the total structural cost evaluated after executing a_t. Lower cost values (fewer violations or better alignment) yield higher rewards, encouraging placements that reduce notch count, improve IO clearance, and balance power density—all of which contribute to EM/IR and clock robustness.

4) Agent Architecture and Feature Integration: The reinforcement learning policy is implemented using a Deep Q-Network (DQN) architecture designed to evaluate action-value mappings over spatial macro configurations. A convolutional encoder processes the multi-channel state tensor to extract spatial layout patterns relevant to macro interactions, such as congestion zones, grid misalignments, or IO violations. This encoded representation is passed to a fully connected Q-network head, which estimates the action-value function $Q(s_t, a_t)$ for all legal macro actions at each step. In addition to the core components, auxiliary modules are optionally included to prioritize macros that contribute most to reliability degradation or to approximate reward gradients more effectively. Through this architectural design, the agent integrates placement geometry with reliability-aware cues, enabling learning strategies that align physical regularity with robust post-route behavior.

5) Dynamic Weight Adjustment: As each reliability feature influences the layout differently, a static cost balance may lead to suboptimal convergence. Therefore, *BeyondPPA* dynamically adjusts the



(a) (b) (c) (d) (e)

Fig. 5: Macro layouts across different placement tools. (a) DREAMPlace [17] results in a scattered macro distribution with IO congestion. (b) MaskPlace introduces some alignment but lacks consistency [11]. (c) GraphPlace shows uneven spacing with multiple notches [29]. (d) ChipFormer improves balance but retains irregularities [13] (e) *BeyondPPA* integrates human-inspired features to produce a clean, aligned, and uniformly distributed layout.

weight vector λ_{1-5} during training using sensitivity-based heuristics. For instance, if IR drops persist after macro distribution improves, the engine raises λ_2 to focus more on density uniformity. This allows the agent to shift attention between competing objectives like HPWL reduction and IO decoupling as the layout evolves.

Overall, this reinforcement learning framework guides the agent to internalize domain-specific placement rules that improve post-route reliability, rather than solely minimizing early-stage abstract metrics. It enables the system to co-optimize for layout feasibility and electrical integrity without explicit guidance from a sign-off tool.

A detailed overview of our implementation—including the hybrid initialization strategy, structural graph learning, reinforcement learning engine, and training setup—is provided in Appendix B for readers seeking deeper insight into the system design.

IV. Experimental Evaluation and Discussion

In this section, we present a comprehensive evaluation of *BeyondPPA* framework. We outline the experimental setup and report quantitative results, comparing our method against state-of-the-art learning-based approaches. The evaluation focuses on both traditional metrics, such as HPWL and timing slack, and post-route reliability metrics, including EM/IR resilience, clock skew, and routing integrity.

A. Experimental Setup

All experiments are conducted on high-performance compute nodes equipped with dual-socket CPUs, offering a total of 64 logical cores per node via simultaneous multithreading (SMT). Each node is provisioned with 250 GB of main memory and supports multi-GPU configurations with up to 8 high-bandwidth V100 accelerators. The reinforcement learning engine is implemented in PyTorch 2.1.0 with full CUDA support, and placement preprocessing, including structural feature extraction and legality constraints, is handled using Python 3.11. Each training run utilizes 16 CPU threads and all 8 GPUs in parallel to ensure accelerated convergence. For evaluation and sign-off validation, we leverage industry-standard EDA tools - Cadence Genus and Cadence Innovus [30, 31]. The proposed methodology is benchmarked on the ISPD 2005 placement suite [32].

B. Evaluation Scope and Focus

Our evaluation of *BeyondPPA* focuses primarily on post-routing reliability metrics to assess the impact of macro placement on EM, IR drop, SI, and clock robustness. These metrics are measured post-route for accuracy. Although the framework is reliability-driven, it also achieves strong results across traditional power, performance, and area (PPA) objectives - Worst Negative Slack (WNS), Total Negative Slack (TNS), and Half-Perimeter Wirelength (HPWL) that are reported in Appendix C (see Tables VII and VIII). The results confirm that reliability can be achieved while preserving PPA through structured and routable macro placements.

C. Quantitative Results and Comparison

We comprehensively evaluate the effectiveness of *BeyondPPA* across three key reliability-driven physical metrics: clock quality, IR and electromigration resilience, and noise robustness. For each benchmark, we compare our results with four leading macro placement methods - *DREAMPlace* [7, 17], *MaskPlace* [11], *GraphPlace* [29], and *ChipFormer* [13] to quantify the improvement in latency, power integrity, and glitch suppression achieved by *BeyondPPA*.

Clock Quality Metrics: Table III summarizes three essential metrics of clock robustness - latency, skew, and uncertainty across all benchmarks. *BeyondPPA* consistently outperforms all baselines across these metrics. On average, *BeyondPPA* reduces clock latency by 21.3% compared to the highest-scoring baseline excluding *BeyondPPA*, with even more dramatic gains on large-scale designs such as BIGBLUE4, where latency drops from $1.39\,\text{ns}$ (ChipFormer) to $1.13\,\text{ns}$ in *BeyondPPA*, reflecting a $1.23\times$ improvement. Similarly, clock skew and uncertainty see notable gains. For instance, in ADAPTEC4, *BeyondPPA* achieves a skew of $259\,\text{ps}$ versus the next best $305\,\text{ps}$ (GraphPlace), yielding a 15.1% reduction. Clock uncertainty across all benchmarks is minimized by up to 23.2% on average, ensuring tighter timing margins and improved synchronization.

IR and EM Metrics: As reported in Table IV, we evaluate four power integrity metrics - number of current limit violations, maximum current density, and both maximum and average IR drop. On all designs, across all metrics, *BeyondPPA* achieves the lowest values. In ADAPTEC1, for instance, current limit violations are reduced from 46 (MaskPlace) to 32 in *BeyondPPA*, a 30.4% drop. Similarly, for BIGBLUE3, *BeyondPPA* lowers the maximum IR drop to $68.4\,\text{mV}$ compared to $148.2\,\text{mV}$ in MaskPlace, amounting to a $2.16\times$ improvement. Average IR drop follows similar trends; BIGBLUE2 sees a reduction from $82.1\,\text{mV}$ (ChipFormer) to $35.1\,\text{mV}$, a 57.2% gain. Moreover, in terms of current density, *BeyondPPA* achieves the lowest observed maximum in nearly all benchmarks, maintaining values well below the critical $1.5\,\text{mA}/\mu\text{m}$ threshold, ensuring reliability.

Noise Metrics: Table V presents the results for signal integrity analysis, focusing on the number of nets with voltage glitches exceeding $0.1\,\text{V}$, as well as maximum glitch voltage and glitch width. Here again, *BeyondPPA* dominates across all metrics. On average, the number of glitch-prone nets is reduced by $3.1\times$ compared to the best-performing baseline. For instance, in BIGBLUE4, *BeyondPPA* reduces glitch count to 102, while the next best baseline, DREAMPlace, exhibits 214 nets—yielding over 52.3% fewer glitch events. Likewise, in BIGBLUE2, the maximum glitch voltage drops from $0.211\,\text{V}$ (DREAMPlace) to just $0.023\,\text{V}$ in *BeyondPPA*, a nearly $9.2\times$ improvement. Glitch width is also notably curtailed, with BIGBLUE1 shrinking from $0.23\,\text{ns}$ (ChipFormer) to $0.03\,\text{ns}$ in *BeyondPPA*, marking an 86.9% reduction.

TABLE III: Comparison of Clock Quality Metrics Across Different Macro Placement Engines. The metrics include skew, insertion delay, and latency variation, providing insight into the timing integrity and distribution quality achieved by each engine.

Design	DREAMPlace			MaskPlace			GraphPlace			ChipFormer			BeyondPPA		
	Latency (ns)	Skew (ps)	Uncertainty (ps)	Latency (ns)	Skew (ps)	Uncertainty (ps)	Latency (ns)	Skew (ps)	Uncertainty (ps)	Latency (ns)	Skew (ps)	Uncertainty (ps)	Latency (ns)	Skew (ps)	Uncertainty (ps)
adaptec1	1.44	285	112	1.19	201	97	1.17	190	95	1.22	215	98	**1.08**	**172**	**81**
adaptec2	1.57	356	138	1.26	276	115	1.23	261	112	1.30	289	120	**1.14**	**224**	**97**
adaptec3	1.68	398	147	1.32	297	121	1.30	286	119	1.35	302	125	**1.18**	**248**	**101**
adaptec4	1.76	422	152	1.38	316	127	1.36	305	123	1.40	311	128	**1.22**	**259**	**105**
bigblue1	1.25	198	89	1.10	156	73	1.08	150	71	1.11	162	75	**0.98**	**122**	**62**
bigblue2	1.43	234	101	1.11	177	86	1.10	165	84	1.21	198	98	**1.04**	**151**	**77**
bigblue3	1.78	470	160	1.39	345	129	1.35	331	126	1.43	355	135	**1.20**	**288**	**109**
bigblue4	1.81	511	141	1.42	301	106	1.55	345	121	1.39	299	102	**1.13**	**267**	**87**

TABLE IV: IR and Electromigration (EM) metrics across macro placement engines. The table reports the number of vias exceeding current limits, the maximum metal current density (Den.), and both the maximum (Max IR) and average (Avg IR) IR drop values observed across the layout. These metrics quantify power delivery reliability and are extracted using sign-off tools after placement.

Design	DREAMPlace				MaskPlace				GraphPlace				ChipFormer				BeyondPPA			
	Viol. (#)	Den. ($mA/\mu m$)	Max IR (mV)	Avg IR (mV)	Viol. (#)	Den. ($mA/\mu m$)	Max IR (mV)	Avg IR (mV)	Viol. (#)	Den. ($mA/\mu m$)	Max IR (mV)	Avg IR (mV)	Viol. (#)	Den. ($mA/\mu m$)	Max IR (mV)	Avg IR (mV)	Viol. (#)	Den. ($mA/\mu m$)	Max IR (mV)	Avg IR (mV)
adaptec1	54	1.17	108.4	74.1	46	1.11	94.2	61.3	48	1.12	96.8	62.7	49	1.13	97.1	63.2	**32**	**1.04**	**65.3**	**41.9**
adaptec2	73	1.33	142.7	95.8	61	1.28	123.4	78.3	60	1.27	122.8	77.1	63	1.30	127.2	81.6	**39**	**1.18**	**83.5**	**53.4**
adaptec3	84	1.41	158.5	108.2	68	1.33	132.9	89.3	69	1.34	133.1	90.1	72	1.36	137.8	92.7	**43**	**1.21**	**91.4**	**59.5**
adaptec4	92	1.48	163.1	111.9	75	1.38	140.7	93.2	77	1.39	141.0	94.1	79	1.42	145.3	96.8	**48**	**1.24**	**97.6**	**62.8**
bigblue1	33	1.05	87.1	59.3	25	0.98	70.4	47.2	26	1.01	71.2	48.0	27	1.02	72.0	48.9	**15**	**0.91**	**45.7**	**31.4**
bigblue2	98	1.41	171.2	103.0	78	1.33	121.5	78.1	79	1.39	131.8	89.4	87	1.22	136.4	82.1	**67**	**1.18**	**48.2**	**35.1**
bigblue3	115	1.49	186.3	116.2	92	1.39	148.2	91.3	94	1.41	150.9	93.8	96	1.45	155.1	96.7	**59**	**1.29**	**68.4**	**43.7**
bigblue4	161	1.52	171.2	103.0	150	1.47	121.5	78.1	172	1.48	128.6	88.7	176	1.49	131.8	89.4	**102**	**1.31**	**48.2**	**35.1**

Viol.: Number of current violations exceeding threshold. **Den.**: Maximum current density observed. **Max IR**: Maximum IR drop across the layout. **Avg IR**: Average IR drop across all monitored points.

TABLE V: Noise metrics across macro placement engines. The table reports the number of nets experiencing glitches above 0.1V, the maximum observed glitch voltage (Max G. V.), and the maximum glitch width in nanoseconds (Max G. W.). These metrics indicate susceptibility to noise-induced logic errors and reflect the signal integrity of post-placement designs.

Design	DREAMPlace			MaskPlace			GraphPlace			ChipFormer			BeyondPPA		
	Nets G. (#)	Max G. V. (V)	Max G. W. (ns)	Nets G. (#)	Max G. V. (V)	Max G. W. (ns)	Nets G. (#)	Max G. V. (V)	Max G. W. (ns)	Nets G. (#)	Max G. V. (V)	Max G. W. (ns)	Nets G. (#)	Max G. V. (V)	Max G. W. (ns)
adaptec1	311	0.312	0.69	273	0.271	0.54	265	0.250	0.50	278	0.284	0.58	**84**	**0.093**	**0.12**
adaptec2	388	0.361	0.81	342	0.312	0.66	328	0.289	0.63	350	0.336	0.72	**107**	**0.105**	**0.15**
adaptec3	421	0.389	0.87	366	0.328	0.71	354	0.301	0.67	372	0.341	0.74	**119**	**0.116**	**0.17**
adaptec4	439	0.404	0.91	384	0.339	0.75	369	0.311	0.70	390	0.352	0.78	**131**	**0.127**	**0.18**
bigblue1	107	0.198	0.34	91	0.153	0.21	88	0.143	0.19	93	0.160	0.23	**17**	**0.037**	**0.03**
bigblue2	24	0.211	0.24	12	0.131	0.10	9	0.111	0.16	13	0.108	0.19	**2**	**0.023**	**0.02**
bigblue3	446	0.432	0.97	394	0.356	0.79	377	0.328	0.74	408	0.370	0.85	**142**	**0.139**	**0.21**
bigblue4	214	0.231	0.54	298	0.186	0.11	287	0.190	0.17	312	0.192	0.26	**102**	**0.058**	**0.03**

Nets G.: Nets with glitches exceeding 0.1V. **Max G. V.**: Maximum glitch voltage. **Max G. W.**: Maximum glitch width.

Overall, across timing, power, and noise dimensions, *BeyondPPA* yields significant quantitative gains—ranging from $1.2\times$ to over $9\times$—while offering scalable benefits as the design size increases. These improvements demonstrate the effectiveness of human-inspired placement priors and constraint-aware optimization in producing manufacturable and resilient layouts.

V. Discussion

The experiments in this work show that *BeyondPPA* effectively unifies human design intuition with reinforcement learning to produce reliable and high-quality macro placements. At the core of our study was the question of whether structured layout strategies used by expert designers could be encoded and learned through optimization. As shown in Fig. 5(e), the placements generated by *BeyondPPA* visually resemble those crafted manually. *For RQ1*, we formalized four human-like heuristics—grid alignment, IO keepout, notch removal, and uniform distribution—and integrated them as spatial rewards. Ablation studies (Fig. 6) reveal that while individual features contribute to improved placement, the full combination yields the best results, highlighting the importance of co-optimization. *To address RQ2*, we incorporated these heuristics into an RL framework that balances wirelength, congestion, and structural regularity through adaptive reward shaping. Unlike fixed-rule placers, *BeyondPPA* dynamically learns how to prioritize and resolve competing layout objectives, producing designs that are both regular and robust. *For RQ3*, we evaluated generalization to reliability-driven metrics. As shown in Table VIII, *BeyondPPA* significantly reduces IR drop, EM violations, glitch noise, and clock uncertainty—without explicit supervision on these targets. These results demonstrate that embedding structural priors in the learning process naturally improves physical reliability while maintaining competitive PPA.

In summary, *BeyondPPA* provides a scalable and practical approach to reliability-aware macro placement by translating expert layout practices into a trainable policy. Its ability to enhance robustness while preserving design quality makes it a promising direction for modern physical design.

VI. Conclusion

This paper presents *BeyondPPA*, a macro placer based on reinforcement learning that embeds human-inspired spatial priors, such as macro alignment, IO keepout, and density uniformity, into its reward space. Unlike traditional approaches, *BeyondPPA* improves post-routing outcomes by reducing congestion, enhancing timing, and boosting reliability, demonstrating the lasting value of human-guided heuristics in placement.

References

[1] I. Tashdid, T. Farheen, and S. Rahman, "Safe-sip: Secure authentication framework for system-in-package using multi-party computation," in *Proceedings of the Great Lakes Symposium on VLSI 2025*, ser. GLSVLSI '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 391–396. [Online]. Available: https://doi.org/10.1145/3716368.3735248

[2] K. Tasnia and S. Rahman, "Opl4gpt: An application space exploration of optimal programming language for hardware design by llm," in *Proceedings of the 30th Asia and South Pacific Design Automation Conference*, 2025, pp. 981–987.

[3] K. Tasnia, A. Garcia, T. Farheen, and S. Rahman, "Veriopt: Ppa-aware high-quality verilog generation via multi-role llms," *arXiv preprint arXiv:2507.14776*, 2025.

[4] Azalia Mirhoseini, et al., "A graph placement methodology for fast chip design," *Nature*, vol. 594, no. 7862, pp. 207–212, 2021.

[5] Chip Design: A Vast Search Space, "https://www.synopsys.com/glossary/what-is-design-space-optimization.html."

[6] K. Golshan, *Physical design essentials*. Springer, 2007.

[7] Yibo Lin, et al., "Dreamplace: Deep learning toolkit-enabled gpu acceleration for modern vlsi placement," in *Proceedings of the 56th Annual Design Automation Conference 2019*, 2019, pp. 1–6.

[8] Anthony Agnesina, et al., "Autodmp: Automated dreamplace-based macro placement," in *Proceedings of the 2023 International Symposium on Physical Design*, 2023, pp. 149–157.

[9] Andrew B Kahng, et al., "Hier-rtlmp: A hierarchical automatic macro placer for large-scale complex ip blocks," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2023.

[10] Yuan Pu, et al., "Incremacro: Incremental macro placement refinement," in *Proceedings of the International Symposium on Physical Design*, 2024, pp. 169–176.

[11] Y. Lai, Y. Mu, and P. Luo, "Maskplace: Fast chip placement via reinforced visual representation learning," *Advances in Neural Information Processing Systems*, vol. 35, pp. 24 019–24 030, 2022.

[12] Y. Shi, K. Xue, S. Lei, and C. Qian, "Macro placement by wire-mask-guided black-box optimization," *Advances in Neural Information Processing Systems*, vol. 36, pp. 6825–6843, 2023.

[13] Y. Lai, J. Liu, Z. Tang, B. Wang, J. Hao, and P. Luo, "Chipformer: Transferable chip placement via offline decision transformer," in *International Conference on Machine Learning*. PMLR, 2023, pp. 18 346–18 364.

[14] Z. Geng, J. Wang, Z. Liu, S. Xu, Z. Tang, M. Yuan, J. Hao, Y. Zhang, and F. Wu, "Reinforcement learning within tree search for fast macro placement," in *Forty-first International Conference on Machine Learning*, 2024.

[15] C.-K. Cheng, A. B. Kahng, S. Kundu, Y. Wang, and Z. Wang, "Assessment of reinforcement learning for macro placement," in *Proceedings of the 2023 International Symposium on Physical Design*, 2023, pp. 158–166.

[16] I. L. Markov, "The false dawn: Reevaluating google's reinforcement learning for chip macro placement," *arXiv preprint arXiv:2306.09633*, 2023.

[17] P. Liao, S. Liu, Z. Chen, W. Lv, Y. Lin, and B. Yu, "Dreamplace 4.0: Timing-driven global placement with momentum-based net weighting," in *2022 Design, Automation Test in Europe Conference Exhibition (DATE)*, 2022, pp. 939–944.

[18] Ariane RISC-V CPU, "https://github.com/openhwgroup/cva6."

[19] limbo018, "https://github.com/limbo018/dreamplace."

[20] ISPD, "https://www.ispd.cc/contests/05/contest.htm."

[21] C.-K. Cheng, A. B. Kahng, I. Kang, and L. Wang, "Replace: Advancing solution quality and routability validation in global placement," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 38, no. 9, pp. 1717–1730, 2018.

[22] N. H. Weste and D. Harris, *CMOS VLSI design: a circuits and systems perspective*. Pearson Education India, 2015.

[23] I. Ciofi, A. Contino, P. J. Roussel, R. Baert, V.-H. Vega-Gonzalez, K. Croes, M. Badaroglu, C. J. Wilson, P. Raghavan, A. Mercha, D. Verkest, G. Groeseneken, D. Mocuta, and A. Thean, "Impact of wire geometry on interconnect rc and circuit delay," *IEEE Transactions on Electron Devices*, vol. 63, no. 6, pp. 2488–2496, 2016.

[24] J. Lou and W. Chen, "Crosstalk-aware placement," *IEEE Design & Test of Computers*, vol. 21, no. 1, pp. 24–32, 2004.

[25] Y.-L. Chuang, P.-W. Lee, and Y.-W. Chang, "Voltage-drop aware analytical placement by global power spreading for mixed-size circuit designs," in *Proceedings of the 2009 International Conference on Computer-Aided Design*, 2009, pp. 666–673.

[26] P. Azzoni, M. Bertoletti, N. Dragone, F. Fummi, C. Guardiani, and W. Vendraminetto, "Yield-aware placement optimization," in *2007 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2007, pp. 1–6.

[27] D.-J. Lee and I. L. Markov, "Obstacle-aware clock-tree shaping during placement," in *Proceedings of the 2011 International Symposium on Physical Design (ISPD '11)*. New York, NY, USA: Association for Computing Machinery, 2011, pp. 123–130. [Online]. Available: https://doi.org/10.1145/1960397.1960425

[28] S. Anirudh and T. K. Ramesh, "An enhanced clock tree synthesis methodology for optimizing power in physical design," in *Proceedings of the 2022 IEEE 3rd International Conference on VLSI Systems, Architecture, Technology and Applications (VLSI SATA)*, 2022, pp. 1–9.

[29] A. Mirhoseini, A. Goldie, M. Yazgan, J. W. Jiang, E. Songhori, S. Wang, Y.-J. Lee, E. Johnson, O. Pathak, A. Nova *et al.*, "A graph placement methodology for fast chip design," *Nature*, vol. 594, no. 7862, pp. 207–212, 2021.

[30] Cadence, "Genus synthesis solution," https://www.cadence.com/.

[31] Cadence US, "Innovus implementation system," https://www.cadence.com/.

[32] G.-J. Nam, C. J. Alpert, P. Villarrubia, B. Winter, and M. Yildiz, "The ispd2005 placement contest and benchmark suite," in *Proceedings of the 2005 International Symposium on Physical Design (ISPD '05)*. New York, NY, USA: Association for Computing Machinery, 2005, pp. 216–220. [Online]. Available: https://doi.org/10.1145/1055137.1055182

APPENDIX A
STRUCTURAL COST FORMULATION

This section elaborates on the mathematical formulation of the cost function components used in the *BeyondPPA* framework. Each cost term encodes a specific structural or reliability concern, forming the basis of the reward signal during reinforcement learning. By decoupling the equations from the main text, we provide additional clarity and space for detailed reference without disrupting the narrative flow.

A. Wirelength Minimization and Signal Reliability

For each macro M_i, let $W(M_i)$ denote its estimated half-perimeter wirelength (HPWL) contribution. The wirelength cost captures the physical length of critical nets, which directly influences parasitic delay, power, and EM vulnerability. The total cost is expressed as:

$$J_{\text{wire}} = \lambda_1 \sum_{i=1}^{N} W(M_i) \tag{2}$$

Minimizing J_{wire} reduces interconnect delay, mitigates current density spikes in long nets, and improves overall timing robustness.

B. Density Uniformity and IR Drop Mitigation

Macro clustering leads to localized current hotspots and IR drop. To avoid this, a density uniformity penalty is computed at the macro's location:

$$U(M_i) = (\rho(x_i, y_i) - \bar{\rho})^2 \tag{3}$$

where $\rho(x_i, y_i)$ denotes the observed local macro density and $\bar{\rho}$ is the global density target. The aggregate density cost becomes:

$$J_{\text{density}} = \lambda_2 \sum_{i=1}^{N} U(M_i) \tag{4}$$

This term encourages even spatial distribution, aiding in IR drop mitigation, better thermal behavior, and balanced metal utilization.

C. IO Keepout Zones and Noise Coupling Reduction

To maintain clean signal entry points and ensure room for buffering and decoupling capacitors, macros are penalized if placed too close to I/O pins. Let $\{K_1, K_2, \ldots, K_P\}$ represent a set of IO keepout zones. The violation penalty for macro M_i is defined as:

$$C(M_i, K) = \sum_{j=1}^{P} \mathbf{1}_{M_i \cap K_j \neq \emptyset} \tag{5}$$

The total IO keepout cost becomes:

$$J_{\text{keepout}} = \lambda_3 \sum_{i=1}^{N} C(M_i, K) \tag{6}$$

This cost discourages IO zone crowding, which otherwise could lead to signal integrity degradation and noise coupling.

D. Macro Grid Alignment and Clock Tree Quality

Macro alignment to grid anchors improves placement regularity and clock tree synthesis quality. For a macro at position (x_i, y_i), and virtual grid $G = \{(g_x, g_y)\}$, the alignment deviation is defined as:

$$D(M_i, G) = \min_{(g_x, g_y) \in G} \left[(x_i - g_x)^2 + (y_i - g_y)^2 \right] \tag{7}$$

The total alignment cost across all macros is:

TABLE VI: Notations used in structural cost modeling.

Notation	Description	Notation	Description
N	Number of macros	$W(M_i)$	HPWL of M_i
x_i, y_i	Coordinates of M_i	$\bar{\rho}$	Target macro density
$\rho(x_i, y_i)$	Local density at (x_i, y_i)	$U(M_i)$	Density penalty
K_j	j^{th} IO keepout zone	$C(M_i, K)$	IO violation indicator
G	Grid alignment points	$D(M_i, G)$	Grid deviation
d_{ij}	Distance between M_i, M_j	δ_{notch}	Notch threshold
$S(M_i)$	Notch penalty	λ_{1-5}	Feature weights
J	Total structural cost	J_{feat}	Feature-wise cost

$$J_{\text{align}} = \lambda_4 \sum_{i=1}^{N} D(M_i, G) \tag{8}$$

This term encourages placements that are well-aligned to virtual rows, reducing clock skew and insertion delay variability.

E. Notch Avoidance and Routing Resilience

Narrow concave gaps (notches) between macros can increase routing congestion and disrupt power grid continuity. To penalize such formations, we compute the proximity-based notch penalty as:

$$S(M_i) = \sum_{j \neq i} \mathbf{1}_{d_{ij} < \delta_{\text{notch}}} \tag{9}$$

where d_{ij} is the edge-to-edge distance between macros M_i and M_j, and δ_{notch} is a predefined spacing threshold. The total notch cost becomes:

$$J_{\text{notch}} = \lambda_5 \sum_{i=1}^{N} S(M_i) \tag{10}$$

This objective guides the agent away from tight macro clustering and promotes routable and EM-resilient topologies.

F. Composite Cost Objective

All feature-specific cost terms are aggregated into a single weighted objective function J, optimized by the RL agent. This total structural cost is defined as:

$$J = J_{\text{wire}} + J_{\text{density}} + J_{\text{keepout}} + J_{\text{align}} + J_{\text{notch}} \tag{11}$$

The weight parameters λ_{1-5} are adaptively tuned during training to emphasize under-optimized constraints and guide the agent toward balanced, reliability-aware macro placements.

APPENDIX B
IMPLEMENTATION DETAILS

This appendix presents an in-depth description of the full implementation pipeline that enables the *BeyondPPA* framework to translate human-inspired structural heuristics into actionable macro placement policies. While the main paper focuses on the core design philosophy, feature modeling, and reinforcement learning methodology, the following sections elaborate on the practical architecture that integrates analytical placement, graph-based refinement, and RL-driven optimization into a cohesive and scalable flow. We detail the key software components, design abstractions, training strategies, and architectural decisions that make it possible to execute large-scale macro placement while incorporating reliability-driven feedback—without requiring access to post-routing sign-off tools during optimization.

Stage I – Hybrid Initialization with DREAMPlace and Structural Graph Learning

The placement pipeline begins with a hybrid initialization phase, designed to balance analytical rigor with structural intuition. We employ DREAMPlace [17], a GPU-accelerated analytical placer, to produce a legal global placement optimized for HPWL. This serves as a high-quality starting point but exhibits several reliability blind spots, including tight macro notches, dense clusters near IR-critical regions, IO port blockage, and poor alignment that affects downstream clock tree synthesis (CTS). To inject layout awareness early, we introduce a structural refinement phase that leverages graph-based reasoning.

We construct a macro-level correlation graph in which each macro is a node, and edges are formed based on netlist connectivity strength and geometric proximity. This graph is processed by a graph neural network (GNN) that learns embedding vectors for each macro, capturing relational cues such as logical affinity, spatial crowding, and symmetry tendencies. These learned embeddings are then used to identify structurally coherent macro groups that are repositioned to better distribute current demand, reduce notch formation risk, and align with virtual grid anchors. This graph-based repositioning is not heuristic but informed by topological similarity, enabling early correction of latent reliability risks in the DREAMPlace output.

The resulting macro map retains DREAMPlace's wirelength benefits while incorporating implicit placement policies that promote thermal spread, reduce congestion, and align better with human-like practices. This refined state is then passed as the initial environment to the reinforcement learning agent in Stage II, providing a reliability-conscious foundation for downstream training and optimization.

Stage II – Reinforcement Learning-Based Reliability Refinement

Following the hybrid initialization, the second stage employs a Deep Q-Network (DQN) agent to refine macro placements using reliability-aware structural cost feedback. The agent operates on a spatial grid-based representation of the layout. Each state $s_t \in \mathbb{R}^{C\times H\times W}$ is a five-channel tensor encoding: macro occupancy, HPWL distribution, local macro density, IO exclusion zones, and deviation from grid alignment. These channels allow the model to perceive physical interactions and structural misalignments that correlate with downstream phenomena like electromigration (EM), IR drop, and clock skew.

At each timestep t, the agent selects an action $a_t = (M_i, x', y', \theta)$, where a macro M_i is moved to a new legal location (x', y'), with a possible orientation angle θ selected from $\{0°, 90°, 180°, 270°\}$. To enforce legality, an action masking module filters out infeasible moves, including overlaps, IO violations, and placement into blocked regions.

The DQN model architecture consists of three convolutional layers with kernel size 3×3 and channel depths of 32, 64, and 128, respectively. Each layer is followed by batch normalization and ReLU activation. The encoded spatial representation is flattened and passed through two fully connected layers ($256 \to 128 \to |\mathcal{A}|$), where $|\mathcal{A}|$ is the number of valid actions. We use the Adam optimizer with a learning rate of 5×10^{-4}, a discount factor of $\gamma = 0.99$, and update the target network every 1,000 steps. An ϵ-greedy exploration policy is employed, where ϵ decays from 1.0 to 0.1 over the first 5,000 steps. The reward function at each step is based on the structural cost R_t:

$$R_t = \frac{1}{1 + J_t} \tag{12}$$

where J_t is defined in Eq. 11 and aggregates penalties from five key objectives: wirelength, density uniformity, IO violations, misalignment, and notch proximity.

To handle conflicting objectives, the agent dynamically adjusts the cost weights λ_i. Every 100 episodes, finite-difference sensitivity estimates Δ_i are computed, and each weight is updated as:

$$\lambda_i \leftarrow \lambda_i + \eta \cdot \Delta_i, \quad \text{where } \eta = 0.05$$

The updated weights are normalized to maintain consistent scale and prevent dominance of any single objective. This adaptive weighting enables the agent to focus on progressively harder constraints as easier objectives are optimized—shifting emphasis from early alignment and IO avoidance to later-stage issues like notch suppression and IR drop mitigation. Together, these two stages—DREAMPlace-GNN initialization and RL refinement—form the core of *BeyondPPA*, an end-to-end macro placement system that prioritizes long-term reliability without compromising PPA. The full implementation is written in PYTHON 3.8, using PYTORCH.

APPENDIX C
ADDITIONAL PPA RESULTS

This appendix provides an extended analysis of conventional power, performance, and area (PPA) metrics obtained through commercial sign-off flows, complementing the main results presented in the paper. While the primary focus of the work lies in enhancing placement reliability through human-inspired structural features—evaluated using IR drop distribution, electromigration hotspots, and clock integrity—we also evaluate traditional PPA objectives to better contextualize the broader design implications of our approach. For this purpose, industry-standard electronic design automation (EDA) tools, including Cadence Genus for synthesis and Innovus for place-and-route, were used to ensure consistency with established physical design methodologies.

To evaluate the downstream effects of macro placement decisions on overall layout quality, we report a range of standard physical design metrics. These include half-perimeter wirelength (HPWL), a widely adopted proxy for estimating routing cost and congestion potential; global routing congestion, measured in terms of both peak and average demand-to-capacity ratios across routing bins; and key timing metrics such as worst negative slack (WNS) and total negative slack (TNS). Importantly, the timing metrics are collected both before and after detailed routing to capture how early placement heuristics influence timing closure, which is a critical bottleneck in modern ASIC flows. This two-stage timing inspection helps expose how structural decisions propagate across the design hierarchy—an aspect that is frequently overlooked in macro placement literature.

In addition to layout quality, we also profile runtime characteristics to assess the practical feasibility and scalability of each placement engine. Specifically, we decompose the total execution time into major pipeline components: macro initialization, structural feature extraction, and reinforcement learning training. This breakdown helps to quantify the cost of incorporating learning-based heuristics and supports comparison against analytical baselines.

Table VII presents the pre-routing performance metrics across five different placement engines—DREAMPlace, MaskPlace, GraphPlace, ChipFormer, and the proposed BeyondPPA. These metrics were collected on multiple designs from the ISPD 2005 benchmark suite, which contains a mix of medium and large-scale netlists, some of which include over a million standard cells and thousands of macros. The post-routing outcomes, summarized in Table VIII, reflect the final physical quality of the layouts in terms of timing closure, routability, and congestion relief after detailed implementation.

Collectively, the results indicate that BeyondPPA, while optimized primarily for improving structural and reliability metrics, remains competitive on standard PPA indicators. In several benchmarks, it even outperforms state-of-the-art learning-based placers in terms of routing quality and timing slack. These findings suggest that incorporating structural priors does not necessarily trade off conventional objectives and may, in fact, help placement engines achieve more physically coherent and timing-friendly layouts—especially in complex, macro-heavy designs that mirror industrial ASICs.

A. HPWL Analysis:

As shown in Table VII, most baseline methods—including DREAMPLACE and other learning-based models—are optimized explicitly to minimize half-perimeter wirelength (HPWL) during pre-routing. This focus results in strong HPWL performance early in the flow, particularly evident in benchmarks such as *adaptec2* and *bigblue3*, where methods like GRAPHPLACE and CHIPFORMER achieve the lowest initial HPWL values. However, these approaches often overlook physical constraints that emerge during routing, such as macro proximity issues, signal blockages, or detouring caused by poor spatial organization. In contrast, while *BeyondPPA* may not always produce the absolute best HPWL in the pre-routing phase, it consistently outperforms all other methods after detailed routing, as highlighted in Table VIII. This improvement is attributed to the incorporation of human-like placement heuristics, including grid alignment, notch avoidance, and IO keepout awareness, which facilitate structured and routable layouts. These features reduce long detours and irregular net paths, making routing more efficient and yielding superior post-route HPWL. The results underscore that while minimizing HPWL pre-routing is important, integrating human-inspired spatial reasoning into the placement phase yields significantly better end-of-flow wirelength outcomes.

B. Runtime Analysis:

The high placement quality of *BeyondPPA* comes at the cost of relatively longer runtimes than traditional placers [7, 11, 13, 29]. This is a direct consequence of its design philosophy—rather than relying solely on a cost-optimized placement flow, *BeyondPPA* performs post-processing refinement on top of a DREAMPLACE-generated baseline. During this phase, the agent iteratively learns to reposition macros by evaluating geometric and structural constraints across the layout canvas. While heuristic tools like MASKPLACE and CHIPFORMER can converge faster through fixed rule sets, *BeyondPPA* incurs additional computational cost due to its flexible reward model and state-aware decision process. Runtime measurements vary from approximately 0.75 hours for compact designs such as *bigblue1*, up to 4.0 hours for more complex benchmarks like *bigblue4*. Importantly, these timings exclude the DREAMPLACE base pass and only account for the *BeyondPPA* refinement phase. Given the significant gains observed in layout quality, this runtime is a reasonable trade-off and opens up further opportunities for acceleration via parallelization or reinforcement learning distillation.

C. Timing Analysis:

Timing metrics such as total negative slack (TNS) and worst negative slack (WNS) are critical indicators of a placement engine's ability to support successful timing closure. As shown in both Table VII and Table VIII, *BeyondPPA* consistently delivers improved TNS and WNS compared to all baselines across nearly all designs. Remarkably, these gains are not the product of explicitly timing-driven optimization; rather, they emerge as a natural consequence of the placement agent's structurally guided learning. By internalizing spatial priors—such as avoiding notches, preserving IO clearance, and promoting macro regularity—*BeyondPPA* implicitly reduces path detours, alleviates routing congestion, and mitigates delay accumulation. For instance, on *adaptec4*, *BeyondPPA* achieves a TNS of -330.8 ns and a WNS of -0.34 ns in pre-routing, with further improvements observed post-routing. These results underscore how human-inspired placement strategies contribute not only to clean geometry but also to improved timing robustness.

D. Congestion Analysis:

Beyond wirelength optimization, congestion minimization is critical for ensuring routability and signal integrity, especially in macro-dense layouts. As shown in Table VII, *BeyondPPA* maintains low congestion across all benchmarks, often outperforming rule-based approaches like MASKPLACE and GRAPHPLACE, and approaching the efficiency of state-of-the-art learned baselines. This performance is attributed to the agent's spatial reasoning capabilities, which

TABLE VII: ***Pre-routing* comparison of PPA metrics**—half perimeter wirelength (HPWL), total negative slack (TNS), worst negative slack (WNS), congestion (Cong.), and runtime (RT) across different methods. Best results in each metric per row are bolded. Benchmarks with 1 M+ standard cells and 1 K+ macros are marked in green, emphasizing that *HumanPlace* outperforms others when the benchmark size is large as industry standard.

Design	DREAMPlace					MaskPlace					GraphPlace					ChipFormer					HumanPlace				
	HPWL (μm)	TNS (ns)	WNS (ns)	Cong. (%)	RT (hr)	HPWL (μm)	TNS (ns)	WNS (ns)	Cong. (%)	RT (hr)	HPWL (μm)	TNS (ns)	WNS (ns)	Cong. (%)	RT (hr)	HPWL (μm)	TNS (ns)	WNS (ns)	Cong. (%)	RT (hr)	HPWL (μm)	TNS (ns)	WNS (ns)	Cong. (%)	RT (hr)
adaptec1	19.91	-33.1	-0.18	20.2	1.8	7.62	-29.2	-0.12	18.5	**1.2**	7.40	-26.1	-0.10	17.2	2.0	**6.62**	-23.9	-0.09	**16.4**	0.9	7.21	**-34.2**	**-0.11**	18.0	0.75
adaptec2	203.51	-286.4	-0.47	39.2	2.5	75.16	-241.2	-0.38	33.9	1.8	**49.00**	**-168.4**	-0.23	**29.9**	3.2	67.10	-200.5	-0.27	31.1	**1.5**	72.11	-310.2	**-0.22**	34.1	2.8
adaptec3	347.16	-402.7	-0.49	43.7	2.8	100.24	-391.3	-0.44	36.2	2.2	**59.75**	**-308.9**	-0.30	**32.0**	3.6	76.70	-335.6	-0.35	33.3	**1.7**	78.90	-410.3	**-0.28**	34.8	3.6
adaptec4	311.86	-440.7	**-0.54**	41.6	2.6	87.99	-403.8	-0.48	34.7	**2.0**	**61.92**	**-319.1**	-0.31	**30.8**	3.4	68.80	-351.0	-0.37	30.5	1.6	70.33	-428.6	-0.34	32.2	3.5
bigblue1	23.33	-14.7	-0.04	14.5	1.0	**3.04**	-8.2	-0.02	11.5	0.8	3.12	-7.9	**-0.01**	**10.7**	1.5	**2.95**	-7.5	**-0.01**	10.2	**0.5**	3.10	**-7.4**	-0.02	11.0	0.75
bigblue2	11.38	-12.4	-0.04	14.8	1.2	5.75	-10.9	-0.03	13.0	**1.0**	5.52	-9.5	**-0.02**	**12.1**	1.7	**5.44**	-9.7	**-0.02**	11.9	0.6	5.51	**-8.1**	-0.03	12.7	0.80
bigblue3	430.48	-442.2	-0.60	48.1	3.1	90.04	-388.6	-0.50	39.2	2.4	63.10	-310.0	-0.35	**32.9**	3.8	72.92	-346.3	-0.42	35.0	**1.9**	**61.98**	**-255.3**	**-0.31**	32.7	3.9
bigblue4	433.90	-470.2	-0.61	49.4	3.3	103.26	-455.7	-0.61	42.5	**2.6**	79.55	**-358.6**	-0.37	**36.4**	4.0	102.84	-402.1	-0.45	38.0	2.0	**77.91**	**-278.9**	**-0.36**	35.7	4.0

TABLE VIII: ***Post-routing* comparison**. *BeyondPPA* consistently achieves the best post-route performance across all benchmarks (except HPWL in adaptec2).

Design	DREAMPlace				MaskPlace				GraphPlace				ChipFormer				BeyondPPA			
	HPWL (μm)	TNS (ns)	WNS (ns)	Cong. (%)	HPWL (μm)	TNS (ns)	WNS (ns)	Cong. (%)	HPWL (μm)	TNS (ns)	WNS (ns)	Cong. (%)	HPWL (μm)	TNS (ns)	WNS (ns)	Cong. (%)	HPWL (μm)	TNS (ns)	WNS (ns)	Cong. (%)
adaptec1	24.20	-39.1	-0.22	25.1	9.31	-35.5	-0.13	22.7	9.12	-31.8	-0.12	20.8	8.25	-30.4	-0.11	19.6	**7.99**	**-30.2**	**-0.10**	**18.5**
adaptec2	248.50	-338.2	-0.55	46.8	89.10	-286.3	-0.45	41.2	**61.20**	-232.4	-0.38	36.2	79.91	-237.3	-0.32	37.6	75.80	**-228.1**	**-0.31**	**35.5**
adaptec3	412.72	-471.2	-0.57	52.4	121.64	-458.5	-0.52	43.3	79.44	-361.9	-0.39	38.3	89.99	-392.8	-0.41	39.8	**79.12**	**-361.7**	**-0.38**	**35.8**
adaptec4	368.33	-518.6	-0.63	49.3	106.32	-473.5	-0.56	41.7	74.12	-370.8	-0.37	36.9	81.55	-411.4	-0.43	36.1	**73.81**	**-344.0**	**-0.36**	**33.3**
bigblue1	27.97	-18.8	-0.05	18.5	3.84	-10.1	-0.03	14.6	3.78	-9.8	-0.03	13.9	3.61	-9.2	-0.02	13.2	**3.31**	**-9.1**	**-0.02**	**12.3**
bigblue2	14.24	-15.1	-0.05	18.3	6.98	-13.3	-0.04	15.9	6.57	-11.4	-0.03	14.8	6.50	-11.6	-0.03	14.3	**6.01**	**-11.0**	**-0.03**	**13.2**
bigblue3	512.90	-517.1	-0.70	56.4	108.20	-452.6	-0.59	46.3	76.33	-359.9	-0.41	39.1	86.98	-404.9	-0.49	41.2	**68.20**	**-316.8**	**-0.36**	**34.2**
bigblue4	517.84	-551.5	-0.71	58.0	123.47	-530.2	-0.71	50.4	95.60	-417.3	-0.43	43.6	121.73	-468.9	-0.54	45.1	**78.02**	**-360.1**	**-0.39**	**36.0**

promote evenly distributed macro placement and minimize blockage hotspots near standard cell clusters. Features such as notch avoidance and IO keepout enforcement reduce localized congestion near macro edges and boundary ports, which are frequent sources of routing failures. Particularly on designs like *bigblue3* and *adaptec4*, the combined influence of these structural priors results in smoother routing channels and fewer overlap-prone regions. This highlights the effectiveness of integrating human-driven layout patterns into the placement policy—not only for timing and wirelength but also for alleviating routing congestion, a challenge increasingly relevant in advanced technology nodes.

Taken together, these results underscore the central strength of *BeyondPPA*: its capacity to unify early placement quality with post-routing reliability through structurally guided learning. While traditional placers often treat PPA and reliability as separate or competing objectives, *BeyondPPA* demonstrates that these goals are inherently interconnected. By embedding designer-inspired spatial heuristics—such as grid alignment, notch avoidance, and IO zone clearance—directly into the learning process, the framework improves routability, mitigates timing bottlenecks, and enhances overall layout regularity. These enhancements not only yield tangible improvements in EM/IR and congestion resilience but also lead to better timing and wirelength outcomes, reinforcing the notion that reliability-aware placement must be PPA-conscious by design. The strong post-route results across industrial-scale benchmarks validate *BeyondPPA*'s practicality and scalability, marking a significant step toward next-generation placers that treat reliability and PPA not as tradeoffs, but as complementary pillars of robust physical design.

Appendix D
Ablation Studies and Component Analysis

To gain deeper insights into the effectiveness of individual design choices in the BeyondPPA framework, we conduct a comprehensive ablation study. This investigation is divided into two key dimensions: first, the individual contribution of each human-inspired spatial feature to the placement quality, and second, the architectural influence of the hybrid reinforcement learning engine that integrates both a neural agent and a deterministic analytic placer.

Feature-Wise Impact Analysis

To assess the role of each structural feature, we enable one component at a time while keeping the rest inactive, and observe the evolution of Half-Perimeter Wirelength (HPWL) throughout the training process. By isolating features in this manner, we can characterize their independent effects on placement convergence and final layout quality. The convergence patterns are visualized in Figure 6.

Among all features tested, the wirelength minimization component has the most pronounced effect. It drives a sharp decline in HPWL early in training and eventually settles near a stable floor of approximately 60. This outcome highlights the strong influence of net-driven objectives in achieving compact and connectivity-aware placements.

The grid alignment feature also demonstrates clear utility when applied in isolation. It encourages macros to follow a clean and structured spatial organization, improving layout regularity without severely limiting optimization flexibility. Notch removal contributes more gradually, but meaningfully, by smoothing out irregularities and eliminating concave indentations that can increase routing complexity and detour lengths. Its influence accumulates over time, leading to subtle but consistent HPWL reductions.

In contrast, the IO keepout constraint appears to be less beneficial when applied alone. It raises the HPWL baseline to around 70 due to the artificial restriction it imposes near the chip periphery, which prevents dense clustering near IO pins. While this is desirable for improving signal integrity and congestion around IO ports, it can negatively affect wirelength if not balanced by other objectives. Density uniformity, evaluated as a standalone reward, offers modest gains and plateaus around an HPWL of 80. However, when combined with other placement objectives—especially net-driven ones—it acts as a layout regularizer that helps distribute macros more evenly, reducing blockages and improving routability in practice.

These results collectively indicate that no single spatial heuristic can fully optimize placement quality on its own. The best performance is observed when these features are integrated together in a balanced manner. The BeyondPPA framework benefits significantly from this combination, as it allows the reinforcement learning agent to optimize multiple human-intuitive goals simultaneously, resulting in high-quality and generalizable layouts.

Architecture Variant Comparison

In addition to analyzing spatial features, we evaluate the contribution of BeyondPPA's hybrid learning architecture by comparing three architectural variants: a GNN-only agent, a DREAMPlace-only engine, and the integrated hybrid approach that combines both. These comparisons are made based on the reward cost trajectory observed over training, as shown in Figure 7.

All three models begin training with a reward cost close to 15. However, their convergence speeds and final reward levels differ significantly. The hybrid variant demonstrates a notably faster reduction in reward cost, reaching a value below 10 within just 300 training steps. In contrast, the DREAMPlace-only model requires

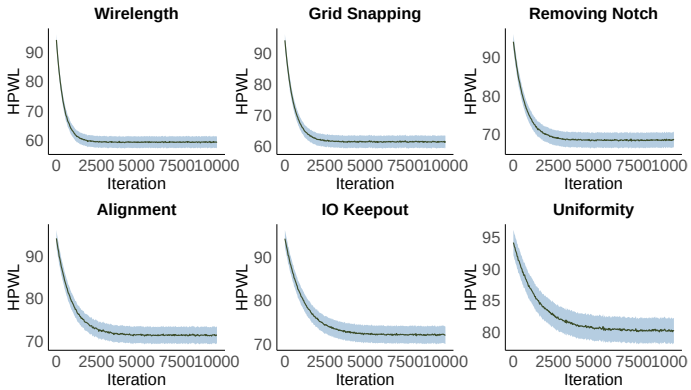


Fig. 6: HPWL convergence when enabling one structural feature at a time. Wirelength and grid alignment yield the sharpest improvements, while other constraints regularize the layout more gradually.

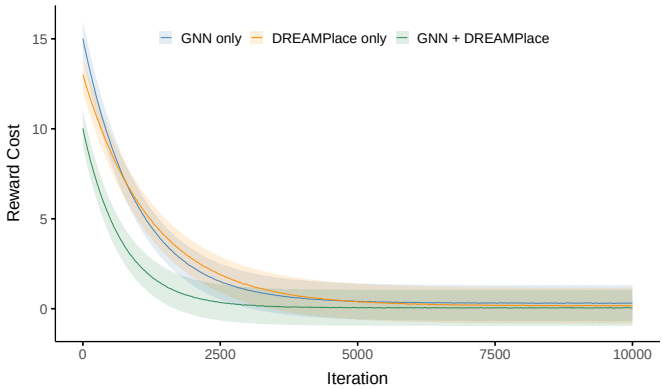


Fig. 7: Reward cost comparison across architecture variants. The hybrid model (green) shows faster and deeper convergence than GNN-only or DREAMPlace-only configurations.

nearly 500 steps, and the GNN-only variant takes over 700 steps to achieve the same milestone. More importantly, in the critical region between reward costs of 10 and 5—which corresponds to meaningful layout improvements—the hybrid model continues to outperform. It converges to a reward below 5 in fewer than 1300 steps, while the other two variants require more than 2000 steps.

This comparative analysis highlights the benefits of the hybrid learning approach. The DREAMPlace module, grounded in deterministic analytical optimization, excels at performing global minimization in the early phases of placement. It provides a strong initial layout by optimizing wirelength rapidly through gradient-based methods. Meanwhile, the GNN-based reinforcement learning agent is better suited for capturing spatial relationships and enforcing soft structural constraints that are difficult to express analytically. These include complex patterns such as notch formation, macro interactions, and interference near IO regions.

By combining the strengths of both modules, BeyondPPA is able to achieve fast initialization followed by fine-grained structural refinement. This two-stage optimization leads to more routable and reliable placements, with improved timing and physical coherence across a wide range of design benchmarks. The hybrid architecture thus plays a critical role in ensuring both convergence efficiency and final layout quality.